Andre marroquin

Laboratorio #5

Preparacion del entorno

In [ ]:
```
# se importan las librerias necesarias
from pathlib import Path
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scapy.all import DNS, IP, Raw, TCP, UDP, rdpcap, sniff, wrpcap
from sklearn.ensemble import IsolationForest

plt.rcParams.update({
    "figure.figsize": (12, 5),
    "axes.grid": True,
    "grid.alpha": 0.3,
    "font.size": 11,
})

ruta_pcap = Path("analisis_paquetes.pcap")
ruta_captura_temporal = Path("captura_prueba_scapy.pcap")

print("Archivo principal:", ruta_pcap)
```

```
Archivo principal: analisis_paquetes.pcap
```

Parte 1 - analisis de paquetes

Testeo de la herramienta

In [19]:
```
# se capturan 10 paquetes y se guardan en un nuevo pcap de prueba
paquetes_prueba = sniff(count=10)
nombre_pcap_prueba = "captura_10_paquetes.pcap"
wrpcap(nombre_pcap_prueba, paquetes_prueba)

print(f"Se capturaron {len(paquetes_prueba)} paquetes y se guardaron en {nombre_pca

pcap_prueba = rdpcap(nombre_pcap_prueba)

print("=== Informacion general del pcap de prueba ===")
print("tipo de dato:", type(pcap_prueba))
print("cantidad de paquetes:", len(pcap_prueba))
print()

print("=== Contenido del pcap ===")
```

```python
for indice_paquete, paquete_actual in enumerate(pcap_prueba, start=1):
    print(f"paquete {indice_paquete}")
    print("tipo de dato del paquete:", type(paquete_actual))
    print("longitud:", len(paquete_actual))
    print("contenido:")
    paquete_actual.show()
    print("-" * 60)

print("\n=== Valores de src_address, dst_address, src_port y dst_port ===")
print(f"{'no.':<5}{'src_address':<20}{'dst_address':<20}{'src_port':<12}{'dst_port'

# se guardan los datos extraidos en una lista
filas_prueba = []

for indice_paquete, paquete_actual in enumerate(pcap_prueba, start=1):
    src_address = paquete_actual[IP].src if IP in paquete_actual else "n/a"
    dst_address = paquete_actual[IP].dst if IP in paquete_actual else "n/a"

    if TCP in paquete_actual:
        src_port = paquete_actual[TCP].sport
        dst_port = paquete_actual[TCP].dport
    elif UDP in paquete_actual:
        src_port = paquete_actual[UDP].sport
        dst_port = paquete_actual[UDP].dport
    else:
        src_port = "n/a"
        dst_port = "n/a"

    filas_prueba.append(
        {
            "src_address": src_address,
            "dst_address": dst_address,
            "src_port": src_port,
            "dst_port": dst_port,
        }
    )

    print(f"{indice_paquete:<5}{src_address:<20}{dst_address:<20}{str(src_port):<12

# se convierte la lista de diccionarios a un dataframe de pandas
tabla_prueba = pd.DataFrame(filas_prueba)
tabla_prueba
```

```
Se capturaron 10 paquetes y se guardaron en captura_10_paquetes.pcap

=== Informacion general del pcap de prueba ===
tipo de dato: <class 'scapy.plist.PacketList'>
cantidad de paquetes: 10

=== Contenido del pcap ===
paquete 1
tipo de dato del paquete: <class 'scapy.layers.l2.Ether'>
longitud: 54
contenido:
###[ Ethernet ]###
  dst       = 48:22:54:68:14:a6
  src       = 4c:49:6c:3a:e4:f4
  type      = IPv4
###[ IP ]###
     version   = 4
     ihl       = 5
     tos       = 0x0
     len       = 40
     id        = 62042
     flags     = DF
     frag      = 0
     ttl       = 128
     proto     = tcp
     chksum    = 0x7b00
     src       = 192.168.68.51
     dst       = 4.249.131.160
     \options   \
###[ TCP ]###
        sport     = 19551
        dport     = https
        seq       = 1986466615
        ack       = 615414530
        dataofs   = 5
        reserved  = 0
        flags     = A
        window    = 252
        chksum    = 0xa9fa
        urgptr    = 0
        options   = []

-------------------------------------------------------------
paquete 2
tipo de dato del paquete: <class 'scapy.layers.l2.Ether'>
longitud: 986
contenido:
###[ Ethernet ]###
  dst       = 4c:49:6c:3a:e4:f4
  src       = 48:22:54:68:14:a6
  type      = IPv4
###[ IP ]###
     version   = 4
     ihl       = 5
     tos       = 0x88
     len       = 972
```

```
     id        = 0
     flags     = DF
     frag      = 0
     ttl       = 53
     proto     = udp
     chksum    = 0xf4ab
     src       = 172.64.155.209
     dst       = 192.168.68.51
     \options   \
###[ UDP ]###
        sport     = https
        dport     = 55888
        len       = 952
        chksum    = 0x122c
###[ Raw ]###
           load      = b'R\x9e\x12Z\x19\xc7\xe7\xb5z%k\xb2\x03\xca|m\xc5:\xde;\x0
f;\xe2\x1f\x1e\x86|\xc3e\xd0,P~E\x12\xaaA\x1a\x9br\xc4\x1e\x0e\x9c\xaeVy\xce\xcd\xcb
3\x17\xf1\x08n\xf0q\xa3~\xb8\xdc\xcf\xddM\x94\xbe\x92\x95\x17\xe8\x18R9\xe2\xbfS\x1f
\x9b\x08\xf2\x98\xb0\x1a)\xd0\xf6E1\x9f\xa8\xb5\xc0\x0b\\\x9f?\xa3\xa8\xf9\xbb\x87o
\x037r\xb5\x9e\xe8\xa4\xb4\x00\xffn\x1a\xd8\xb7\xdd\xd3\x1f\xbcWhP`\xc1)\xa1\xa8\'\x
19\xa8\xcb\x8f\xdc\x02\xdc\rY\xe7\xb1,\xe0\xb4p\xe5\x13\xff6\'W\r\x1da9\xe7\x80\xc4
\x95\xa7\x81f\x8d\x15\xb5\x93\x1c\xe4\xf5\x17\x01\xcd\xc1\xa5\xf8\x1a\xa8\x8b\xca\xc
dR\x9e,7\x18\t\xcc\xd9\xd1p\xa2S\x83\x92l\x1d\xf3\xdf\x81\x98\xe9$\x9bVb\xb6]P\x82\x
ea\x9c#\xbf9\x85\xc3\xcd\xee\xd8\x89y\x9b\xa8#\x8b\x19\x9f/\xcc.\xf6\xc0\xdd\xe2\xb5
\xbc\x83\x99\xb0\xd8\xfb\xd7\xe3\x12\xd5;W\xe7e\x80\xf71/\xd3\x06\xf1V\xff\xebX\x8f
\xecN\xa4\x83@\x99Z\xf9i\xd5\x08f\xfd~yr\x90|\x99\xacO\x1d\xee\xb6S-\xb1I\x8b\xa7Q\x
0b_\xbazO\x80\xa4N\xa8W\x16\x1b\x9a~\xdc\x97\xbe;3\xae\x89\xb5Q\x92\xff\xc9v\x89\x84
\x8dmL\xdeg\x1b\xa3[d1D\x0c"\x06\x15\xfe8\xbej\xa8\xf7\xfb\x0b\x9c\x95\'l\xf8\xfa6w
\xcb\nn!<"\xb2G\x0fQ\xb0Q^\xae\x15X}\x96\xed\x8a.\xd8\x8f\xd5\x82\xcc!\x138\xec\x11
\x13\xda\xf4\xb9\xf0hOb\x15\xb3\r\x84\xb1\x1ck\xdc\x8b[\x9caw.\x9f\xe7W\x03\xa4H\x90
\xd6=S,\x16\xaa$\xcb\xd11\xce+\xcc\xeaS\r\xab|\xaf\x14\xd4\xd4\xcbvK\xbcZn>\xe6\x8f
\xf8\xf00\xda\xa4R&\xa6VN\xd3\x97A\xf5f\xbb]Xa\xd1a\xc05Vhx\xee\x95\xfa\xaf\xa0V\xc8
\xfc\xf3K\xfek\x92\x80\xa5\xe5\xd9^\x18g\xb8p \xe7D(\x94=\xdf\xdf($\x7f\x0fS\xa6\xe5
\xfc)\x8dh\xd9q\x1a,\xb8\xe2\x99\xbciI/\x02Z\xaa\xa4\x90/t^?\r\x0f\x87\xe5\xfe\xa4*K
pT\x97\xb2\xf5\x84\x88[T\xbbHY.\xc7N\xf3W\x8f\xf6\x1f\x93h`\xe9~!\x91\xba\xf2D\xa5\x
d6\xc0 \xc3\xbe\x12<b\x97\x90\x84\xa8zs\xb0?\x8d\xc7N^\xc3\xf6\xcf\xa6\xdfY\x04TV\x9
f\xf9\x8f\x15\xbd\x0cUqn\xfc1f\xc4\x84\xe5\xd5\xa1\xb1/5\xd7)@\\\xf1\x95\xdc\x01\xf2
\x15\x03\xcd/<\x9a\x8a\xfb\x11\xf8\x93k\x19\x7fX\xcc\xa9\xaaA\x02Pd\xf3\x8c\x94Z\x12
\x83f_h\xe7\x91\x1b\xeb\xaaI\x93L\xa2>[,\xbf\x8a\xe7\xb1w\xe7\x7f\xd5g\xc2\x07\xd9~
\xcfK\xf2\xb0D95K\xb0\x98o\x94\x88Mj\xb9\x01\x05\xe7\x0e\\}\xd8\x1e\x1aB\xbf\x06n\x1
2\xd2\xfc\x9cea\x0c8cB\xe7\xcb]4\x02\x83(\xbe\x82X\xf6\xd6\xe6\x11\xf7\xfb\xa1\x19\x
cb\xc9\x0boJ9\x18d\xe2\xcb\xbaA\x9c\x98Z$\x93\x88\xd1\x8c,\xd4\x84\xbf16\xef\xd2\xbe
\xed\x15`\xf1v\ty\xb4)\xb8NG\x10|q\x87X.l\xbc\xd8\xe6}xu\xde\xd0A\x1f#\xf0A\xd5<\xf6
^c\xa5\xbau\xa5\xb2\x80\x93\xf0\x8a\xe7\xb9%\x16\xcd\xf2\xf2q\xdf\xa7?<\xe4hQ\xdd0!e
\xaa\x92\xd0\xa1UR\rE\x86\x9a%l\x1c\x84\xaeF\x98\x0bw\xf7?\x99\x9e\xe1\x1c\x9d\xf
8:\x86\xa1\x7f1:\x90Gh\xe1\xa8\xfel\xb7]C\xd1:\x87\xd2\xfd_V\xfc\xdc\xd5bn\x8b\x12\x
feL\x03\x1e\xf7DwS\nzp^\xb4\xb7]\xa4\xbb\x12LU%o\xc2\x87\xf8\x81\xe8\xed\x83\xfb\\\x
d6\xa3\x94\x9e\xc0\x9d\xd4(G\x96g\xff\x81Y\x05\\I\xce\x02\xcc\x96\x0b\xf7'

--------------------------------------------------------------
paquete 3
tipo de dato del paquete: <class 'scapy.layers.l2.Ether'>
longitud: 87
contenido:
###[ Ethernet ]###
  dst       = 48:22:54:68:14:a6
```

```
  src       = 4c:49:6c:3a:e4:f4
  type      = IPv4
###[ IP ]###
     version   = 4
     ihl       = 5
     tos       = 0x0
     len       = 73
     id        = 16393
     flags     = DF
     frag      = 0
     ttl       = 128
     proto     = udp
     chksum    = 0x6dad
     src       = 192.168.68.51
     dst       = 172.64.155.209
     \options   \
###[ UDP ]###
        sport     = 55888
        dport     = https
        len       = 53
        chksum    = 0xab83
###[ Raw ]###
           load      = b'D\x01\x12\x8c}\xa1\x03\x04\xa7\x8e\x12X}\xaf\x03\x16\xca"\x
edtf\x8d\xa2\x91\r\x02\xa0W*)!S\xaf\x0f\xf2)\xcc\xeb\xea\xcep\xca\x1d\xdcx'

------------------------------------------------------------
paquete 4
tipo de dato del paquete: <class 'scapy.layers.l2.Ether'>
longitud: 489
contenido:
###[ Ethernet ]###
  dst       = 4c:49:6c:3a:e4:f4
  src       = 48:22:54:68:14:a6
  type      = IPv4
###[ IP ]###
     version   = 4
     ihl       = 5
     tos       = 0x0
     len       = 475
     id        = 47259
     flags     = DF
     frag      = 0
     ttl       = 40
     proto     = tcp
     chksum    = 0xb0d
     src       = 4.249.131.160
     dst       = 192.168.68.51
     \options   \
###[ TCP ]###
        sport     = https
        dport     = 19551
        seq       = 615414530
        ack       = 1986466615
        dataofs   = 5
        reserved  = 0
        flags     = PA
```

```
        window    = 1650
        chksum    = 0xc8b3
        urgptr    = 0
        options   = []
###[ Raw ]###
           load      = b'\x17\x03\x03\x01\xae\xaf\x8f\xed\xa1*\x15\xec\x05\xc7\xb9\x
11\xce"\xbd\xcc\x08\x02\xf6\t\xe4\xcc5,d\xf7j\xb3\x89\x0c\xdbi\xea\x1b_)\xa0\x86\xa8
\xd5\xcc\x8d\xe8\x8a\xf5)\xcc\xaa\xa5\xc3\x9cP\xe4/\xf2C\xcd\xd6\xdd\x89\xa8\xfd\x1
3;\xc7\xbc\xdd+&\xf4\xbc:\xff86\x17\xc10\x8f\xf9\x1f\xb8Z\xf4\xa3\xd17\x10kW\xc6\x83
\xc6 \xb2\xc2\xcc](9\x13\x81\nB\x03?V \xd3\x0f\x00\xb5\x0b7\xde\xa7\x92P]\xd6d\xbe@
\x12\x030\xe6H\x93M\xc4#\x93_B\xa6\x19\xe8\x87#\xe7 \x82.\xc3\xc6\x1d\x908\xea\x05\x
8azCm\xd0\xc1L\x1c~yC\xaaU\x18\'5\x80L\x80\xf7\x1e\x11\xce\x01\x1c\xdf\x02\xfc\x991^
\x83O:\x85\xfcLz\xd1\xad#\x86\x190\xda\x0c\xe7\xc0\x01\xb4\xdd\'\xdfD\xaa\x02\x93\xd
8x\xa95\xb2JW\xa6\x1cS}\xbc\xd8ir\x02n;Y\xbe\x8c\xf0\xcc\xb4\x97\x08b\xde\x00A\xc8\x
ad\x100\x1a\x95\xc7\\k?\xa1\x10\xbf\xda\x01\xf3Mj\x9e\xce&4\xc7X~C.X\xcbw\xc9\x08\x9
7Z\xf3>A\xf6eD\xaa\x1b\xf5\x19\xfeK]\x96\xdc\xd8\xef5\xe0\x8cdJ4\xc5\xb5\xd6U]\xceiG
\x0f$\xe4\xc9\n\xf8\xe7P9o\xcc\xfa)\xf3\xf5\x91\xc8\xc5.\xc7|\x1fr\x11SP=\xe7\x86-\x
15\x8c\xb6B\xb8\xb4\'\x05k"\xaf\xa0d;#X\x12C\x15\xa5\xcbe."?39\x87\xacI\xd0Nm\x14\xa
e\xb7\xadUh\t\x9d\x90\xcf(\xdc\xd6\xcc\xaa\xac\xbb\x98*\xd0_\x8f\x9f\xad!\xf8\xb2=)
\xb6\xa3\x00En\xbb]\xef\xdc\xb4{\xbe\xcb\x89\x1a\x84F_{\xd7~\x8e\xf6L\xff\xf6`\x9c7&
^P\xe8\xc9\xe6\x83'

------------------------------------------------------------
paquete 5
tipo de dato del paquete: <class 'scapy.layers.l2.Ether'>
longitud: 139
contenido:
###[ Ethernet ]###
  dst       = 4c:49:6c:3a:e4:f4
  src       = 48:22:54:68:14:a6
  type      = IPv4
###[ IP ]###
     version   = 4
     ihl       = 5
     tos       = 0x0
     len       = 125
     id        = 47260
     flags     = DF
     frag      = 0
     ttl       = 40
     proto     = tcp
     chksum    = 0xc6a
     src       = 4.249.131.160
     dst       = 192.168.68.51
     \options   \
###[ TCP ]###
        sport     = https
        dport     = 19551
        seq       = 615414965
        ack       = 1986466615
        dataofs   = 5
        reserved  = 0
        flags     = PA
        window    = 1650
        chksum    = 0x327b
        urgptr    = 0
```

```
        options   = []
###[ Raw ]###
           load      = b'\x17\x03\x03\x00P\x1c\xc3\xfd{\xa0\xc9^\x1d\xf62?l\x06f\xec
\xb8\xc1#\xaa\xffC\xca\x15<\xf3\xee\x8c \x8e\xf9$WS\xf29\xbd\x04\x01\xf1\x88\x83\xab
I\xafg\xa9\x7f\xdc?\xdd\x00\xdc\x1f)\x11\xd4\xe7\xbbMc\x8cT\xbabu(h\t\xea\x9e\xe0>\x
c8\xf6zD\xbd\x02\xed\xa2'

------------------------------------------------------------
paquete 6
tipo de dato del paquete: <class 'scapy.layers.l2.Ether'>
longitud: 54
contenido:
###[ Ethernet ]###
  dst       = 48:22:54:68:14:a6
  src       = 4c:49:6c:3a:e4:f4
  type      = IPv4
###[ IP ]###
     version   = 4
     ihl       = 5
     tos       = 0x0
     len       = 40
     id        = 62043
     flags     = DF
     frag      = 0
     ttl       = 128
     proto     = tcp
     chksum    = 0x7aff
     src       = 192.168.68.51
     dst       = 4.249.131.160
     \options   \
###[ TCP ]###
        sport     = 19551
        dport     = https
        seq       = 1986466615
        ack       = 615415050
        dataofs   = 5
        reserved  = 0
        flags     = A
        window    = 250
        chksum    = 0xa7f4
        urgptr    = 0
        options   = []

------------------------------------------------------------
paquete 7
tipo de dato del paquete: <class 'scapy.layers.l2.Ether'>
longitud: 151
contenido:
###[ Ethernet ]###
  dst       = 4c:49:6c:3a:e4:f4
  src       = 48:22:54:68:14:a6
  type      = IPv4
###[ IP ]###
     version   = 4
     ihl       = 5
     tos       = 0x0
```

```
     len       = 137
     id        = 47261
     flags     = DF
     frag      = 0
     ttl       = 40
     proto     = tcp
     chksum    = 0xc5d
     src       = 4.249.131.160
     dst       = 192.168.68.51
     \options   \
###[ TCP ]###
        sport     = https
        dport     = 19551
        seq       = 615415050
        ack       = 1986466615
        dataofs   = 5
        reserved  = 0
        flags     = PA
        window    = 1650
        chksum    = 0xa652
        urgptr    = 0
        options   = []
###[ Raw ]###
           load      = b'\x17\x03\x03\x00\\R\x19\x065\x04E\x97\xb3\x03B6\xea:\xea\x9
3\xb0B\xd2|\x93\x85\xd5\xcdie \x04u\xbatCi\xd5\xd0\xf3\x7f\x1d\x8e\xc8\xd8\xce3}\xae
\xc1Og\xb3\xc8\xbe\x07\x82t\x06\x12\x1a\xfdY\xbb"\x11\x8bJ\xb3\xb5\x94\xd1ZB0\xc6\xb
2ZjO\xb4\x19\x14\xac+\xcc*\xa3 \xc7\xdf\x82Y\x18)\xe5\xc6'

--------------------------------------------------------------
paquete 8
tipo de dato del paquete: <class 'scapy.layers.l2.Ether'>
longitud: 141
contenido:
###[ Ethernet ]###
  dst       = 4c:49:6c:3a:e4:f4
  src       = 48:22:54:68:14:a6
  type      = IPv4
###[ IP ]###
     version   = 4
     ihl       = 5
     tos       = 0x0
     len       = 127
     id        = 47262
     flags     = DF
     frag      = 0
     ttl       = 40
     proto     = tcp
     chksum    = 0xc66
     src       = 4.249.131.160
     dst       = 192.168.68.51
     \options   \
###[ TCP ]###
        sport     = https
        dport     = 19551
        seq       = 615415147
        ack       = 1986466615
```

```
        dataofs   = 5
        reserved  = 0
        flags     = PA
        window    = 1650
        chksum    = 0x465c
        urgptr    = 0
        options   = []
###[ Raw ]###
           load      = b'\x17\x03\x03\x00R\x9a+"p\xe73\x9a\xb3{My\x85\x10\xfe\x7fr\x
16\x92\xc3\x87\x95\xb4\xf6\x9a\xbb\x0f\xe3\x1e\x12\x9aV\x9d\x87Z\x95o\xae\xb7G\xd3.^
\xa97\xaf\xe6\xb9\xc6cPcF\x98\xc9+C\x04\xd9\x94\x1dK\x05^|%\xb1Q\xbf\t\x8aj\x8c\x11g
\xd1,RqH\x9d\xa0{'

------------------------------------------------------------
paquete 9
tipo de dato del paquete: <class 'scapy.layers.l2.Ether'>
longitud: 54
contenido:
###[ Ethernet ]###
  dst       = 48:22:54:68:14:a6
  src       = 4c:49:6c:3a:e4:f4
  type      = IPv4
###[ IP ]###
     version   = 4
     ihl       = 5
     tos       = 0x0
     len       = 40
     id        = 62044
     flags     = DF
     frag      = 0
     ttl       = 128
     proto     = tcp
     chksum    = 0x7afe
     src       = 192.168.68.51
     dst       = 4.249.131.160
     \options   \
###[ TCP ]###
        sport     = 19551
        dport     = https
        seq       = 1986466615
        ack       = 615415234
        dataofs   = 5
        reserved  = 0
        flags     = A
        window    = 255
        chksum    = 0xa737
        urgptr    = 0
        options   = []

------------------------------------------------------------
paquete 10
tipo de dato del paquete: <class 'scapy.layers.l2.Ether'>
longitud: 99
contenido:
###[ Ethernet ]###
  dst       = 4c:49:6c:3a:e4:f4
```

```
  src       = 48:22:54:68:14:a6
  type      = IPv4
###[ IP ]###
     version   = 4
     ihl       = 5
     tos       = 0x0
     len       = 85
     id        = 47263
     flags     = DF
     frag      = 0
     ttl       = 40
     proto     = tcp
     chksum    = 0xc8f
     src       = 4.249.131.160
     dst       = 192.168.68.51
     \options   \
###[ TCP ]###
        sport     = https
        dport     = 19551
        seq       = 615415234
        ack       = 1986466615
        dataofs   = 5
        reserved  = 0
        flags     = PA
        window    = 1650
        chksum    = 0xeef5
        urgptr    = 0
        options   = []
###[ Raw ]###
           load      = b'\x17\x03\x03\x00(#\xd6Vv1T\xa6\xea\x90=\xe3\xfaE9\xc2We\xc9
z\x94\xf4f\xffi\xee\xf7\xfd\xc9\x9cV\x05\xdd\xb0\xaf\x834 !\x10\xf2'

------------------------------------------------------------

=== Valores de src_address, dst_address, src_port y dst_port ===
no.  src_address         dst_address         src_port    dst_port
1    192.168.68.51       4.249.131.160       19551       443
2    172.64.155.209      192.168.68.51       443         55888
3    192.168.68.51       172.64.155.209      55888       443
4    4.249.131.160       192.168.68.51       443         19551
5    4.249.131.160       192.168.68.51       443         19551
6    192.168.68.51       4.249.131.160       19551       443
7    4.249.131.160       192.168.68.51       443         19551
8    4.249.131.160       192.168.68.51       443         19551
9    192.168.68.51       4.249.131.160       19551       443
10   4.249.131.160       192.168.68.51       443         19551
```

Out[19]:

	src_address	dst_address	src_port	dst_port
0	192.168.68.51	4.249.131.160	19551	443
1	172.64.155.209	192.168.68.51	443	55888
2	192.168.68.51	172.64.155.209	55888	443
3	4.249.131.160	192.168.68.51	443	19551
4	4.249.131.160	192.168.68.51	443	19551
5	192.168.68.51	4.249.131.160	19551	443
6	4.249.131.160	192.168.68.51	443	19551
7	4.249.131.160	192.168.68.51	443	19551
8	192.168.68.51	4.249.131.160	19551	443
9	4.249.131.160	192.168.68.51	443	19551

Carga del archivo real y conversion a dataframe

Estadisticas Y deteccion

In [ ]:

```
# se identifica el protocolo de cada paquete
def extraer_protocolo(paquete):
    if paquete.haslayer(DNS):
        return "DNS"
    if paquete.haslayer(TCP):
        return "TCP"
    if paquete.haslayer(UDP):
        return "UDP"
    if paquete.haslayer(IP):
        return "IP"
    return "OTRO"

# se obtienen los puertos solo si el paquete usa tcp o udp
def puertos_si_existen(paquete):
    if paquete.haslayer(TCP):
        return int(paquete[TCP].sport), int(paquete[TCP].dport)
    if paquete.haslayer(UDP):
        return int(paquete[UDP].sport), int(paquete[UDP].dport)
    return None, None

# se extrae el payload en bytes si existe una capa raw
def obtener_payload_bytes(paquete):
    if paquete.haslayer(Raw):
        return bytes(paquete[Raw].load)
    return b""

# se construtye una fila con los datos utiles del paquete
def fila_desde_paquete(indice, paquete):
```

```python
    src_port, dst_port = puertos_si_existen(paquete)
    payload_bruto = obtener_payload_bytes(paquete)
    return {
        "indice": indice,
        "src_address": paquete[IP].src if paquete.haslayer(IP) else None,
        "dst_address": paquete[IP].dst if paquete.haslayer(IP) else None,
        "src_port": src_port,
        "dst_port": dst_port,
        "payload_len": len(payload_bruto),
        "pkt_len": len(bytes(paquete)),
        "protocolo": extraer_protocolo(paquete),
        "payload_hex": payload_bruto.hex(),
        "payload_ascii": payload_bruto.decode("utf-8", errors="replace"),
        "resumen": paquete.summary(),
    }

# se carga el archivo pcap real y construir un DataFrame con los datos de cada paqu
pcap_real = rdpcap(str(ruta_pcap))
df = pd.DataFrame([fila_desde_paquete(indice, paquete) for indice, paquete in enume

print("Cantidad total de paquetes:", len(df))
print("Columnas disponibles:", list(df.columns))

display(df.head(10))
display(df["protocolo"].value_counts().rename_axis("protocolo").reset_index(name="f
```

```
Cantidad total de paquetes: 62
Columnas disponibles: ['indice', 'src_address', 'dst_address', 'src_port', 'dst_por
t', 'payload_len', 'pkt_len', 'protocolo', 'payload_hex', 'payload_ascii', 'resume
n']
```

	indice	src_address	dst_address	src_port	dst_port	payload_len	pkt_len	protocolo	
0	0	10.1.10.53	84.54.22.33	53	53	905	975	DNS	efbfb
1	1	84.54.22.33	10.1.10.53	53	53	0	98	DNS	
2	2	10.1.10.53	84.54.22.33	53	53	919	989	DNS	
3	3	84.54.22.33	10.1.10.53	53	53	0	98	DNS	
4	4	10.1.10.53	84.54.22.33	53	53	956	1026	DNS	e
5	5	84.54.22.33	10.1.10.53	53	53	0	98	DNS	
6	6	10.1.10.53	84.54.22.33	53	53	942	1012	DNS	e
7	7	84.54.22.33	10.1.10.53	53	53	0	98	DNS	
8	8	10.1.10.53	84.54.22.33	53	53	947	1017	DNS	e
9	9	84.54.22.33	10.1.10.53	53	53	0	98	DNS	

	protocolo	frecuencia
0	DNS	62

In [ ]:
```
# se obtienen las ips origen y destino mas frecuentes
top_src = df["src_address"].dropna().value_counts().rename_axis("src_address").rese
top_dst = df["dst_address"].dropna().value_counts().rename_axis("dst_address").rese

ip_origen_frecuente = top_src.loc[0, "src_address"]
ip_destino_frecuente = top_dst.loc[0, "dst_address"]

# se obtienen las ips destino con las que se comunica la ip origen mas frecuente
destinos_de_origen = (
    df[df["src_address"] == ip_origen_frecuente]["dst_address"]
```

```python
    .dropna()
    .value_counts()
    .rename_axis("dst_address")
    .reset_index(name="frecuencia")
)

# se obtienen los puertos destino usados por la ip origen mas frecuente
puertos_destino_origen = (
    df[df["src_address"] == ip_origen_frecuente]["dst_port"]
    .dropna()
    .astype(int)
    .value_counts()
    .rename_axis("dst_port")
    .reset_index(name="frecuencia")
)

# se obtienen los puertos origen usados hacia la ip destino mas frecuente
puertos_origen_destino = (
    df[df["dst_address"] == ip_destino_frecuente]["src_port"]
    .dropna()
    .astype(int)
    .value_counts()
    .rename_axis("src_port")
    .reset_index(name="frecuencia")
)

display(top_src)
display(top_dst)
display(destinos_de_origen)
display(puertos_destino_origen)
display(puertos_origen_destino)
```

	src_address	frecuencia
0	10.1.10.53	31
1	84.54.22.33	29
2	75.75.75.75	2

	dst_address	frecuencia
0	10.1.10.53	31
1	84.54.22.33	29
2	75.75.75.75	2

	dst_address	frecuencia
0	84.54.22.33	29
1	75.75.75.75	2

	dst_port	frecuencia
0	53	31

	src_port	frecuencia
0	53	31

Respuestas

a. Muestre cual es la IP origen mas frecuente

Respuesta: Con base en los resultados obtenidos, la IP origen mas frecuente es 10.1.10.53 con 31 paquetes.

b. Muestre cual es la IP destino mas frecuente

Respuesta: La IP destino mas frecuente tambien es 10.1.10.53 con 31 paquetes.

c. A que IPs se comunica la IP del inciso a

Respuesta: La IP 10.1.10.53 se comunica principalmente con 84.54.22.33 (29), 75.75.75.75 (2). La conversacion dominante es con 84.54.22.33.

d. A que puertos destino se comunica la IP del inciso a

Respuesta: En esta traza la IP 10.1.10.53 se comunica al puerto destino 53 (31).

e. A que puertos origen se comunica la IP del inciso b

Respuesta: Cuando 10.1.10.53 aparece como destino, recibe trafico desde los puertos origen 53 (31).

f. Indique el proposito de los puertos que mas aparecen en los incisos d y e

Respuesta: El puerto 53 se usa normalmente para DNS, es decir, para resolver nombres de dominio. Lo llamativo aqui es que en la conversacion principal aparecen 53 -> 53 y ademas hay payloads binarios grandes.

Z-score

In [ ]:
```
# se calcula la media y desviacion estandar del payload
media_payload = df["payload_len"].mean()
desviacion_payload = df["payload_len"].std(ddof=0)
```

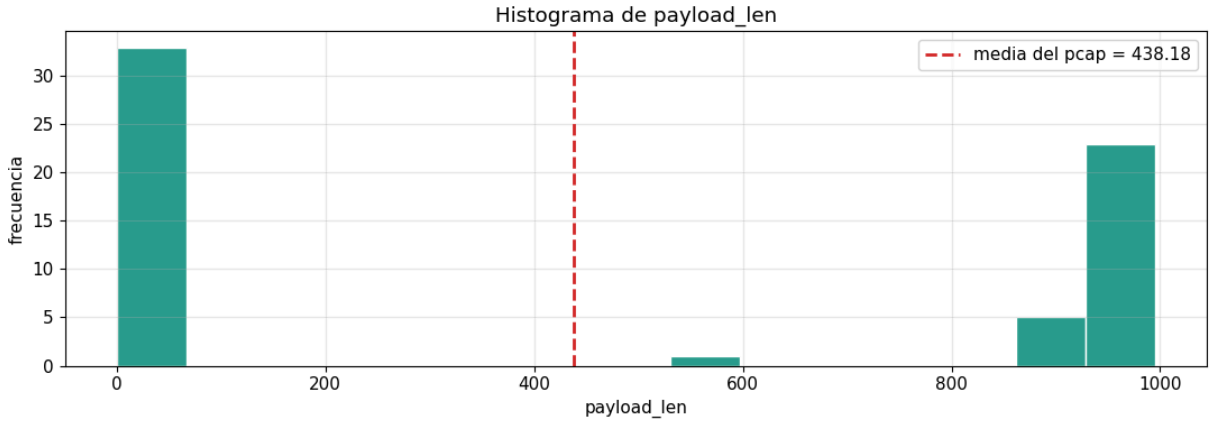
```python
if desviacion_payload == 0:
    df["z_score_payload"] = 0.0
else:
    df["z_score_payload"] = (df["payload_len"] - media_payload) / desviacion_payloa

# se calcula el z score usando valores de referencia para dns
media_dns = 50.0
desviacion_dns = 15.0
df["z_score_dns"] = (df["payload_len"] - media_dns) / desviacion_dns

# se filtran paquetes anomalos segun distintos umbrales
anomalias_z_2 = df[df["z_score_payload"].abs() > 2].copy()
anomalias_z_3 = df[df["z_score_payload"].abs() > 3].copy()
anomalias_dns = df[df["z_score_dns"].abs() > 2].copy()

print("Media de payload_len:", round(media_payload, 2))
print("Desviacion estandar de payload_len:", round(desviacion_payload, 2))
print("Cantidad con |Z| > 2:", len(anomalias_z_2))
print("Cantidad con |Z| > 3:", len(anomalias_z_3))
print("Cantidad con z-score DNS por conocimiento de dominio:", len(anomalias_dns))

display(anomalias_z_2[["indice", "src_address", "dst_address", "src_port", "dst_por
display(anomalias_z_3[["indice", "src_address", "dst_address", "src_port", "dst_por
display(anomalias_dns[["indice", "src_address", "dst_address", "src_port", "dst_por

plt.figure(figsize=(11, 4))
plt.hist(df["payload_len"], bins=15, color="#2a9d8f", edgecolor="white")
plt.axvline(media_payload, color="#d62828", linestyle="--", linewidth=2, label=f"me
plt.title("Histograma de payload_len")
plt.xlabel("payload_len")
plt.ylabel("frecuencia")
plt.legend()
plt.tight_layout()
plt.show()
```

```
Media de payload_len: 438.18
Desviacion estandar de payload_len: 470.62
Cantidad con |Z| > 2: 0
Cantidad con |Z| > 3: 0
Cantidad con z-score DNS por conocimiento de dominio: 62
```

indice	src_address	dst_address	src_port	dst_port	payload_len	pkt_len	protocolo

indice	src_address	dst_address	src_port	dst_port	payload_len	pkt_len	protocolo

	indice	src_address	dst_address	src_port	dst_port	payload_len	pkt_len	protocolo
0	0	10.1.10.53	84.54.22.33	53	53	905	975	DNS
1	1	84.54.22.33	10.1.10.53	53	53	0	98	DNS
2	2	10.1.10.53	84.54.22.33	53	53	919	989	DNS
3	3	84.54.22.33	10.1.10.53	53	53	0	98	DNS
4	4	10.1.10.53	84.54.22.33	53	53	956	1026	DNS
5	5	84.54.22.33	10.1.10.53	53	53	0	98	DNS
6	6	10.1.10.53	84.54.22.33	53	53	942	1012	DNS
7	7	84.54.22.33	10.1.10.53	53	53	0	98	DNS
8	8	10.1.10.53	84.54.22.33	53	53	947	1017	DNS
9	9	84.54.22.33	10.1.10.53	53	53	0	98	DNS
10	10	10.1.10.53	84.54.22.33	53	53	989	1059	DNS
11	11	84.54.22.33	10.1.10.53	53	53	0	98	DNS
12	12	10.1.10.53	84.54.22.33	53	53	952	1022	DNS
13	13	84.54.22.33	10.1.10.53	53	53	0	98	DNS
14	14	10.1.10.53	84.54.22.33	53	53	923	993	DNS
15	15	84.54.22.33	10.1.10.53	53	53	0	98	DNS
16	16	10.1.10.53	84.54.22.33	53	53	903	973	DNS
17	17	84.54.22.33	10.1.10.53	53	53	0	98	DNS
18	18	10.1.10.53	84.54.22.33	53	53	936	1006	DNS
19	19	84.54.22.33	10.1.10.53	53	53	0	98	DNS



Respuestas

a. Calcule el Z-score de la columna payload para todos los paquetes. Hay paquetes con |Z| > 2 o |Z| > 3? Muestre los resultados. Encontro paquetes anomalos?

Respuesta: No. Usando la media y la desviacion estandar del propio pcap, la media de payload_len fue 438.18 y la desviacion estandar fue 470.62. Con esos valores no aparecieron paquetes con |Z| > 2 ni con |Z| > 3, por lo que este z-score no detecto paquetes anomalos.

b. Muestre un histograma de payload_len. La distribucion es normal? Si el Z-score no detecto anomalías, explique por que la forma de la distribucion causa este problema.

Respuesta: La distribucion no se ve normal. Es claramente bimodal: un grupo queda en 0 bytes y otro se concentra cerca de 900 a 1000 bytes. Como la media y la desviacion se calculan sobre esos dos grupos mezclados, el z-score termina absorbiendo el comportamiento raro dentro de la propia referencia y por eso no marca outliers.

c. Ahora calcule el Z-score usando conocimiento de dominio: el tamano tipico de una consulta DNS es de aproximadamente 50 bytes con una desviacion estandar de 15 bytes. Utilice estos valores como la media y desviacion de referencia en lugar de calcularlos del pcap. Que paquetes se detectan ahora como anomalos? Cuantos paquetes fueron detectados?

Respuesta: Usando como referencia una consulta DNS tipica de 50 bytes con desviacion estandar de 15 bytes, se obtuvieron 62 paquetes fuera del rango esperado con |Z| > 2. Sin embargo, los paquetes mas sospechosos son los enviados de 10.1.10.53 hacia 84.54.22.33, ya que presentan payload_len de aproximadamente 905 a 989 bytes, valores demasiado altos para una consulta DNS normal. Los paquetes con payload_len = 0 tambien quedan fuera del rango, pero deben interpretarse con precaucion porque en este analisis payload_len corresponde a la capa Raw, no necesariamente al tamano total del mensaje DNS.

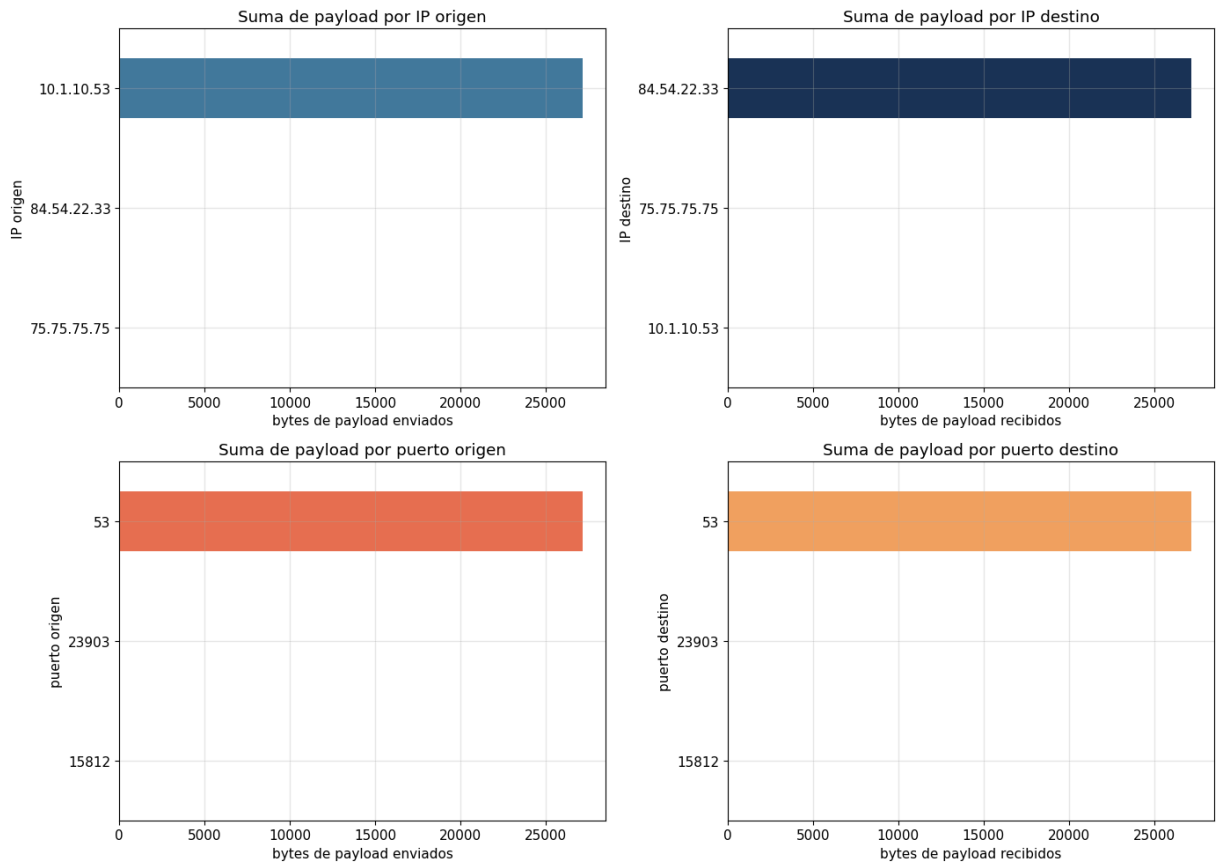
d. Que puede decir que nos ensena esto sobre la importancia de conocer el o los protocolos a analizar al aplicar tecnicas de deteccion de anomalias?

Respuesta: El conocimiento del protocolo cambia por completo la interpretacion. Si solo se usa la estadistica interna del pcap, el trafico raro se normaliza. En cambio, cuando se compara contra lo esperable para DNS, el trafico resalta. Tambien no basta con conocer el protocolo, tambien hay que elegir bien la variable que se esta midiendo.

Graficas

In [16]:

```
# se muestran las diferentes graficas de la suma de payloads por src, dst, sport y
suma_por_src = df.groupby(df["src_address"].fillna("sin_ip"))["payload_len"].sum().
suma_por_dst = df.groupby(df["dst_address"].fillna("sin_ip"))["payload_len"].sum().
suma_por_sport = df.groupby(df["src_port"].fillna(-1).astype(int))["payload_len"].s
suma_por_dport = df.groupby(df["dst_port"].fillna(-1).astype(int))["payload_len"].s

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

suma_por_src.plot(kind="barh", ax=axes[0, 0], color="#457b9d")
axes[0, 0].set_title("Suma de payload por IP origen")
axes[0, 0].set_xlabel("bytes de payload enviados")
axes[0, 0].set_ylabel("IP origen")

suma_por_dst.plot(kind="barh", ax=axes[0, 1], color="#1d3557")
axes[0, 1].set_title("Suma de payload por IP destino")
axes[0, 1].set_xlabel("bytes de payload recibidos")
axes[0, 1].set_ylabel("IP destino")

suma_por_sport.plot(kind="barh", ax=axes[1, 0], color="#e76f51")
axes[1, 0].set_title("Suma de payload por puerto origen")
axes[1, 0].set_xlabel("bytes de payload enviados")
axes[1, 0].set_ylabel("puerto origen")

suma_por_dport.plot(kind="barh", ax=axes[1, 1], color="#f4a261")
axes[1, 1].set_title("Suma de payload por puerto destino")
axes[1, 1].set_xlabel("bytes de payload recibidos")
axes[1, 1].set_ylabel("puerto destino")

plt.tight_layout()
plt.show()
```



Deteccion automatica con isolation forest

In [ ]:

```
# se calcula la tasa de contaminacion a partir de las anomalias dns
contaminacion = len(anomalias_dns) / len(df)
if contaminacion <= 0:
    contaminacion_modelo = max(1 / len(df), 0.001)
else:
    contaminacion_modelo = min(contaminacion, 0.5)

modelo_if = IsolationForest(
    n_estimators=200,
    contamination=contaminacion_modelo,
    random_state=42,
)

# se marcan las filas detectadas como anomalias
df["isolation_forest"] = modelo_if.fit_predict(df[["payload_len", "pkt_len"]])
df["anomalia_if"] = df["isolation_forest"] == -1
anomalias_if = df[df["anomalia_if"]].copy()

print("Contaminacion original:", round(contaminacion, 4))
print("Contaminacion ajustada para el modelo:", round(contaminacion_modelo, 4))
print("Cantidad de anomalias detectadas por Isolation Forest:", len(anomalias_if))

display(anomalias_if[["indice", "src_address", "dst_address", "src_port", "dst_port
```

```
Contaminacion original: 1.0
Contaminacion ajustada para el modelo: 0.5
Cantidad de anomalias detectadas por Isolation Forest: 30
```

	indice	src_address	dst_address	src_port	dst_port	payload_len	pkt_len	protocolo
0	0	10.1.10.53	84.54.22.33	53	53	905	975	DNS
2	2	10.1.10.53	84.54.22.33	53	53	919	989	DNS
4	4	10.1.10.53	84.54.22.33	53	53	956	1026	DNS
6	6	10.1.10.53	84.54.22.33	53	53	942	1012	DNS
8	8	10.1.10.53	84.54.22.33	53	53	947	1017	DNS
10	10	10.1.10.53	84.54.22.33	53	53	989	1059	DNS
12	12	10.1.10.53	84.54.22.33	53	53	952	1022	DNS
14	14	10.1.10.53	84.54.22.33	53	53	923	993	DNS
16	16	10.1.10.53	84.54.22.33	53	53	903	973	DNS
18	18	10.1.10.53	84.54.22.33	53	53	936	1006	DNS
20	20	10.1.10.53	84.54.22.33	53	53	995	1065	DNS
22	22	10.1.10.53	75.75.75.75	15812	53	0	79	DNS
23	23	10.1.10.53	75.75.75.75	23903	53	0	79	DNS
24	24	75.75.75.75	10.1.10.53	53	15812	0	119	DNS
25	25	75.75.75.75	10.1.10.53	53	23903	0	131	DNS
28	28	10.1.10.53	84.54.22.33	53	53	930	1000	DNS
30	30	10.1.10.53	84.54.22.33	53	53	948	1018	DNS
32	32	10.1.10.53	84.54.22.33	53	53	981	1051	DNS
34	34	10.1.10.53	84.54.22.33	53	53	948	1018	DNS
36	36	10.1.10.53	84.54.22.33	53	53	983	1053	DNS
38	38	10.1.10.53	84.54.22.33	53	53	981	1051	DNS
40	40	10.1.10.53	84.54.22.33	53	53	979	1049	DNS
42	42	10.1.10.53	84.54.22.33	53	53	954	1024	DNS
48	48	10.1.10.53	84.54.22.33	53	53	957	1027	DNS
50	50	10.1.10.53	84.54.22.33	53	53	963	1033	DNS
52	52	10.1.10.53	84.54.22.33	53	53	949	1019	DNS
54	54	10.1.10.53	84.54.22.33	53	53	961	1031	DNS
56	56	10.1.10.53	84.54.22.33	53	53	971	1041	DNS
58	58	10.1.10.53	84.54.22.33	53	53	913	983	DNS
60	60	10.1.10.53	84.54.22.33	53	53	532	602	DNS

Respuestas

a. Utilice las columnas numericas del DataFrame (payload_len y pkt_len) para entrenar un modelo de Isolation Forest con un valor igual al numero de paquetes detectados en el inciso 4c dividido por el numero de paquetes totales para definir su valor para el parametro contamination. Que paquetes son marcados como anomalías?

Respuesta: El valor de contaminacion calculado fue 1.0, pero Isolation Forest no permite usar un valor tan alto, ya que solo acepta valores entre 0 y 0.5. Por eso fue necesario ajustarlo a 0.5 para poder ejecutar el modelo. Con esp, el modelo marco 30 paquetes como anomalos. La mayoria pertenecen a la conversacion entre 10.1.10.53 y 84.54.22.33, que es justamente donde aparecen los payloads mas grandes y fuera de lo comun. Tambien se marcaron algunos paquetes del intercambio con 75.75.75.75.

b. Las anomalias detectadas coinciden con las IPs o conversaciones sospechosas que identifico con los Z-score? Explique.

Respuesta: Si, coinciden en parte. Isolation Forest vuelve a resaltar la conversacion con 84.54.22.33, que ya habia aparecido como la mas sospechosa en el analisis con z-score. Sin embargo, no la separa de forma totalmente limpia, porque tambien incluye algunos paquetes de otro intercambio DNS que parecen menos relevantes. Esto muestra que el modelo sirve para orientar la investigacion y señalar donde vale la pena mirar, pero aun asi hace falta revisar manualmente el trafico para confirmar que realmente es sospechoso.

Investigacion del payload

In [ ]:
```
# se filtran solo las conexiones cuya ip origen es la mas frecuente
df_origen_frecuente = df[df["src_address"] == ip_origen_frecuente].copy()

# se agrupan las conexiones por origen destino y payload para ver repeticiones y by
df_payload_agrupado = (
    df_origen_frecuente.groupby(["src_address", "dst_address", "payload_hex"], drop
    .agg(["count", "sum"])
    .reset_index()
    .rename(columns={"count": "repeticiones", "sum": "bytes_totales"})
    .sort_values(["bytes_totales", "repeticiones"], ascending=False)
)

# se clacilan con que ip se intercambiaron mas bytes en total
intercambio_bytes = (
    df[
        (df["src_address"] == ip_origen_frecuente)
        | (df["dst_address"] == ip_origen_frecuente)
```

```
    ]
    .assign(
        ip_contraparte=lambda datos: np.where(
            datos["src_address"] == ip_origen_frecuente,
            datos["dst_address"],
            datos["src_address"],
        )
    )
    .groupby("ip_contraparte", dropna=False)["payload_len"]
    .sum()
    .sort_values(ascending=False)
    .rename_axis("ip_contraparte")
    .reset_index(name="payload_total")
)

# se toman la ip con mayor intercambio de bytes como ip sospechosa
ip_sospechosa = intercambio_bytes.loc[0, "ip_contraparte"]

# se filtran la conversacion entre la ip origen mas frecuente y la ip sospechosa
df_conversacion = df[
    (
        (df["src_address"] == ip_origen_frecuente)
        & (df["dst_address"] == ip_sospechosa)
    )
    | (
        (df["src_address"] == ip_sospechosa)
        & (df["dst_address"] == ip_origen_frecuente)
    )
].copy()

payloads_conversacion = [bytes.fromhex(payload_hex) for payload_hex in df_conversac
primer_payload = payloads_conversacion[0]

# se p[repara] una vista hexadecimal y una vista ascii del primer payload
def vista_hexadecimal(datos, limite=48):
    return " ".join(f"{byte:02x}" for byte in datos[:limite])
primer_payload_hex = vista_hexadecimal(primer_payload, limite=48)
primer_payload_ascii = primer_payload[:200].decode("utf-8", errors="replace")

# se crea un resumen auxiliar de paquetes dns para facilitar la inspeccion
df_resumen_dns = pd.DataFrame(
    [
        {
            "indice": indice,
            "summary": paquete.summary(),
            "src_address": paquete[IP].src if paquete.haslayer(IP) else None,
            "dst_address": paquete[IP].dst if paquete.haslayer(IP) else None,
            "src_port": int(paquete[UDP].sport) if paquete.haslayer(UDP) else (int(
            "dst_port": int(paquete[UDP].dport) if paquete.haslayer(UDP) else (int(
            "payload_len": len(bytes(paquete[Raw].load)) if paquete.haslayer(Raw) e
        }
        for indice, paquete in enumerate(pcap_real)
    ]
)

display(df_origen_frecuente.head())
```

```
display(df_payload_agrupado.head(10))
display(intercambio_bytes)
display(df_conversacion.head(10))
display(
    df_resumen_dns[
        (df_resumen_dns["src_address"].isin([ip_origen_frecuente, ip_sospechosa, "7
        & (df_resumen_dns["dst_address"].isin([ip_origen_frecuente, ip_sospechosa,
    ].head(12)
)

print("IP sospechosa:", ip_sospechosa)
print("Cantidad de filas en la conversacion:", len(df_conversacion))
print("Cantidad de payloads no vacios:", len(payloads_conversacion))
print("Primer payload en hexadecimal:")
print(primer_payload_hex)
print("\nPrimer payload con decodificacion segura:")
print(primer_payload_ascii)
```

	indice	src_address	dst_address	src_port	dst_port	payload_len	pkt_len	protocolo	
0	0	10.1.10.53	84.54.22.33	53	53	905	975	DNS	efbfb
2	2	10.1.10.53	84.54.22.33	53	53	919	989	DNS	
4	4	10.1.10.53	84.54.22.33	53	53	956	1026	DNS	e
6	6	10.1.10.53	84.54.22.33	53	53	942	1012	DNS	e
8	8	10.1.10.53	84.54.22.33	53	53	947	1017	DNS	e

	src_address	dst_address	payload_hex	repeticiones
26	10.1.10.53	84.54.22.33	efbfbdefbfbdefbfbd3c3e7defbfbdefbfbdefbfbd25ef...	1
14	10.1.10.53	84.54.22.33	77efbfbd20efbfbd0252efbfbd1900efbfbd5e7fefbfbd...	1
2	10.1.10.53	84.54.22.33	1d39efbfbd15c6b8efbfbd0a63efbfbdefbfbd7b1befbf...	1
3	10.1.10.53	84.54.22.33	2a0befbfbdcbaaefbfbd66efbfbd75efbfbdefbfbdefbf...	1
7	10.1.10.53	84.54.22.33	4fefbfbdefbfbdefbfbdefbfbdc880efbfbdefbfbdefbf...	1
24	10.1.10.53	84.54.22.33	efbfbdefbfbd58efbfbdefbfbd09efbfbdf3bc89b7efbf...	1
20	10.1.10.53	84.54.22.33	efbfbd4befbfbd65efbfbd34efbfbd50244d1171efbfbd...	1
6	10.1.10.53	84.54.22.33	4c0c3eefbfbd372b2befbfbd727defbfbdefbfbd587fef...	1
11	10.1.10.53	84.54.22.33	72cc90207e16efbfbd00efbfbdefbfbd7fefbfbd02efbf...	1
17	10.1.10.53	84.54.22.33	7f30efbfbd3672efbfbd16efbfbd1e6142efbfbdefbfbd...	1

	ip_contraparte	payload_total
0	84.54.22.33	27167
1	75.75.75.75	0

	indice	src_address	dst_address	src_port	dst_port	payload_len	pkt_len	protocolo	
0	0	10.1.10.53	84.54.22.33	53	53	905	975	DNS	efbfb
1	1	84.54.22.33	10.1.10.53	53	53	0	98	DNS	
2	2	10.1.10.53	84.54.22.33	53	53	919	989	DNS	
3	3	84.54.22.33	10.1.10.53	53	53	0	98	DNS	
4	4	10.1.10.53	84.54.22.33	53	53	956	1026	DNS	e
5	5	84.54.22.33	10.1.10.53	53	53	0	98	DNS	
6	6	10.1.10.53	84.54.22.33	53	53	942	1012	DNS	e
7	7	84.54.22.33	10.1.10.53	53	53	0	98	DNS	
8	8	10.1.10.53	84.54.22.33	53	53	947	1017	DNS	e
9	9	84.54.22.33	10.1.10.53	53	53	0	98	DNS	

	indice	summary	src_address	dst_address	src_port	dst_port	payload_len
0	0	Ether / IP / UDP / DNS Qry b'google.com.' / Raw	10.1.10.53	84.54.22.33	53	53	905
1	1	Ether / IP / UDP / DNS Ans 2607:f8b0:4005:807:...	84.54.22.33	10.1.10.53	53	53	0
2	2	Ether / IP / UDP / DNS Qry b'google.com.' / Raw	10.1.10.53	84.54.22.33	53	53	919
3	3	Ether / IP / UDP / DNS Ans 2607:f8b0:4005:807:...	84.54.22.33	10.1.10.53	53	53	0
4	4	Ether / IP / UDP / DNS Qry b'google.com.' / Raw	10.1.10.53	84.54.22.33	53	53	956
5	5	Ether / IP / UDP / DNS Ans 2607:f8b0:4005:807:...	84.54.22.33	10.1.10.53	53	53	0
6	6	Ether / IP / UDP / DNS Qry b'google.com.' / Raw	10.1.10.53	84.54.22.33	53	53	942
7	7	Ether / IP / UDP / DNS Ans 2607:f8b0:4005:807:...	84.54.22.33	10.1.10.53	53	53	0
8	8	Ether / IP / UDP / DNS Qry b'google.com.' / Raw	10.1.10.53	84.54.22.33	53	53	947
9	9	Ether / IP / UDP / DNS Ans 2607:f8b0:4005:807:...	84.54.22.33	10.1.10.53	53	53	0
10	10	Ether / IP / UDP / DNS Qry b'google.com.' / Raw	10.1.10.53	84.54.22.33	53	53	989
11	11	Ether / IP / UDP / DNS Ans 2607:f8b0:4005:807:...	84.54.22.33	10.1.10.53	53	53	0

```
IP sospechosa: 84.54.22.33
Cantidad de filas en la conversacion: 58
Cantidad de payloads no vacios: 29
Primer payload en hexadecimal:
ef bf bd 50 4e 47 0d 0a 1a 0a 00 00 00 0d 49 48 44 52 00 00 01 62 00 00 00 ef bf bd
08 06 00 00 00 28 ef bf bd 54 52 00 00 3a ef bf bd 49 44 41

Primer payload con decodificacion segura:

IHDR b (TR: IDATx |T;K
E$ Zi-;N2V VaZ!ik[.okeP\]-dg

```

Respuestas

a. Cree un nuevo DataFrame que incluya unicamente las conexiones con la direccion IP origen mas frecuente

Respuesta: Se creo `df_origen_frecuente` filtrando solo las filas con `src_address == "10.1.10.53"`.

b. Obtenga un nuevo DataFrame con las columnas Src Address, Dst Address y agrupelas por payload

Respuesta: Se genero `df_payload_agrupado`, que resume origen, destino, `payload_hex`, repeticiones y bytes acumulados. Eso permite ver que la mayor parte del volumen sale hacia **84.54.22.33**.

c. Obtenga la IP que mas ha intercambiado bytes con la IP mas frecuente. Confirme que esta es la misma IP sospechosa identificada por las tecnicas automaticas.

Respuesta: La IP que mas bytes intercambia con 10.1.10.53 es 84.54.22.33, con 27167 bytes de payload. Si, coincide con la conversacion que resaltaron el z-score con conocimiento de dominio y Isolation Forest.

d. Cree un nuevo DataFrame con la conversacion entre la IP mas frecuente y la IP sospechosa

Respuesta: Se construyo `df_conversacion` filtrando ambos sentidos entre 10.1.10.53 y 84.54.22.33.

e. Obtenga los payloads del DataFrame anterior y añada cada uno en un array

Respuesta: Los payloads se guardaron en la lista `payloads_conversacion`.

f. Muestre el contenido del primer array

Respuesta: El primer payload se muestra en hexadecimal y en una decodificacion segura. Los primeros bytes observados fueron:

```
ef bf bd 50 4e 47 0d 0a 1a 0a 00 00 00 0d 49 48 44 52 00 00 01 62 00 00
00 ef bf bd 08 06 00 00 00 28 ef bf bd 54 52 00 00 3a ef bf bd 49 44 41
```

Y la vista textual segura empieza asi:

```
``
``
`
`
IHDR b (TR :IDATx |T;K;E
3wf<_ Y r y
`
`
8c ew
`
`
iaK u}Ţw d "
`
`
&9Ɛ
`
`
`

## g. Examine los primeros bytes del contenido. Encuentra algun dato que no haga sentido que se envie a traves del puerto explicado en el inciso 3f? En que consiste el ataque detectado? Por que es importante combinar tecnicas automaticas con investigacion manual del payload?

**Respuesta:** Si. En los primeros bytes aparece informacion propia de un archivo PNG, lo cual no tiene sentido dentro de trafico DNS normal por el puerto 53. Eso sugiere que el canal DNS se esta usando para transportar datos binarios que no deberian ir ahi. Con base en esa evidencia, lo mas probable es que se trate de un caso de tunelizacion por DNS. Esta parte manual fue importante, porque las tecnicas automaticas ayudaron a señalar donde estaba el trafico raro, pero solo al revisar el contenido real del payload se pudo entender mejor que tipo de informacion se estaba moviendo y por que era sospechosa.
```